Parrot Detection

Species and Keypoints using Yolo8

by Cătălina Chirteș and Raul Moiș



Inspiration for the project



Characteristics of this parrot

Name: Winnie

Age: 9 months

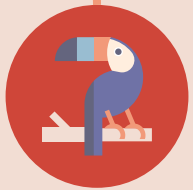
Species: Cockatiel

What was our goal?

Detect parrots in videos

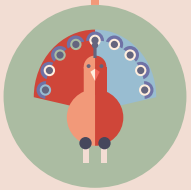
1st step

Detect both if it is a parrot, and if known, what species



2nd step

Detect some anatomically relevant to bird keypoints



3rd step

Identify what the parrot is doing based on the keypoints



How we achieved it



01 **Research**

Investigating existing techniques

02 **Dataset**

Looking for a dataset that could be useful

03 **Experiments**

Training and testing different models

04 **Evaluation**

Comparing and deciding the best model

Dataset selection

There were no datasets
that could be directly used

01 Researched for datasets but the options were limited

02

03 We had to create one of our own, combining two existing datasets



05 Obtaining a dataset of 2512 labelled parrot images (2213/299)

04 Combined it with 'Parrot Species Detection Computer Vision Dataset' by Bruno Lee from Roboflow Universe

06 Downloaded ‘Open Images Dataset V7’ subsample of parrot images for detection

Data collection

Open Images Dataset V7

One class ‘Parrot’ → 1490 (1414/76)

Parrot Species Detection Computer Vision Dataset

17 classes, each one a different parrot species → 1022 (799/223)

Parrot Hierarchical Detection Dataset

Relabelled all images with 2 overlapping bounding boxes (having the same coordinates)

1st

the species label

2nd

the general parrot label

A glance at the dataset

YOLO11 format

- train (images, labels)
- test (images, labels)
- data.yaml

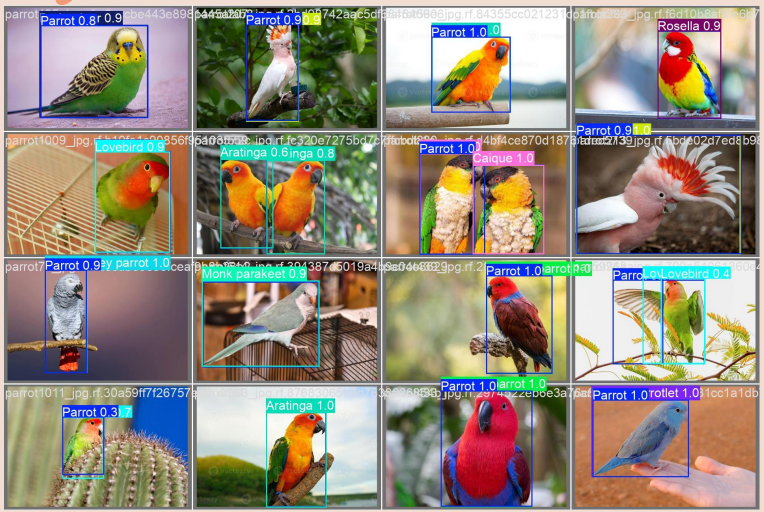
Published on Kaggle

https://www.kaggle.com/datasets/catalinaadelachirtes/parrot-hierarchical-detection-dataset



0: Parrot
1: African grey parrot
2: Amazon parrot
3: Aratinga
4: Budgerigar
5: Caique
6: Cockatiel
7: Cockatoo
8: Eclectus parrot
9: Green cheek conure
10: Kakariki
11: Lorikeet
12: Lovebird
13: Macaw
14: Monk parakeet
15: Pacific parrotlet
16: Rosella
17: Senegal parrot

Initial experiments



01

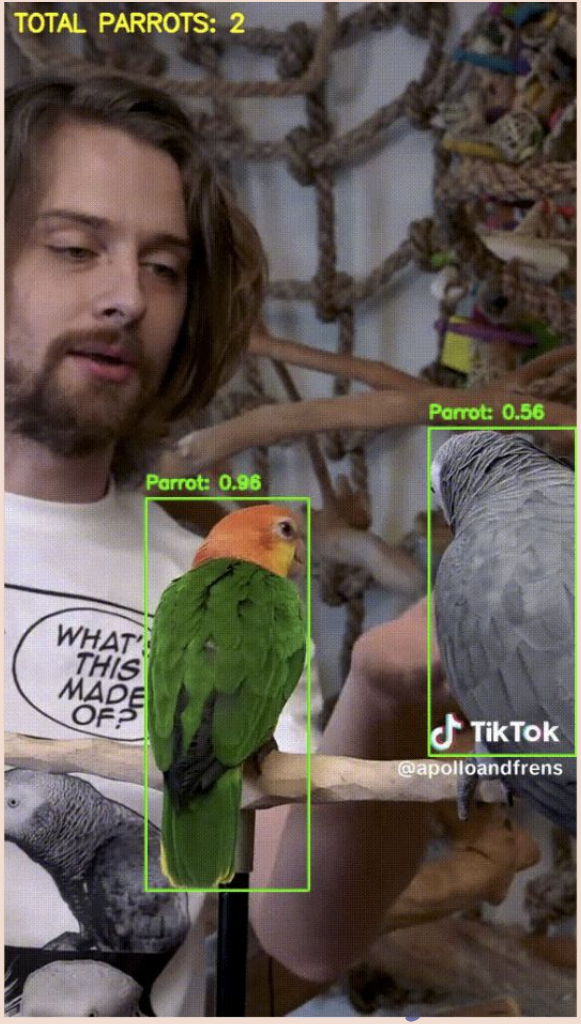
Trained a YOLOv11 model

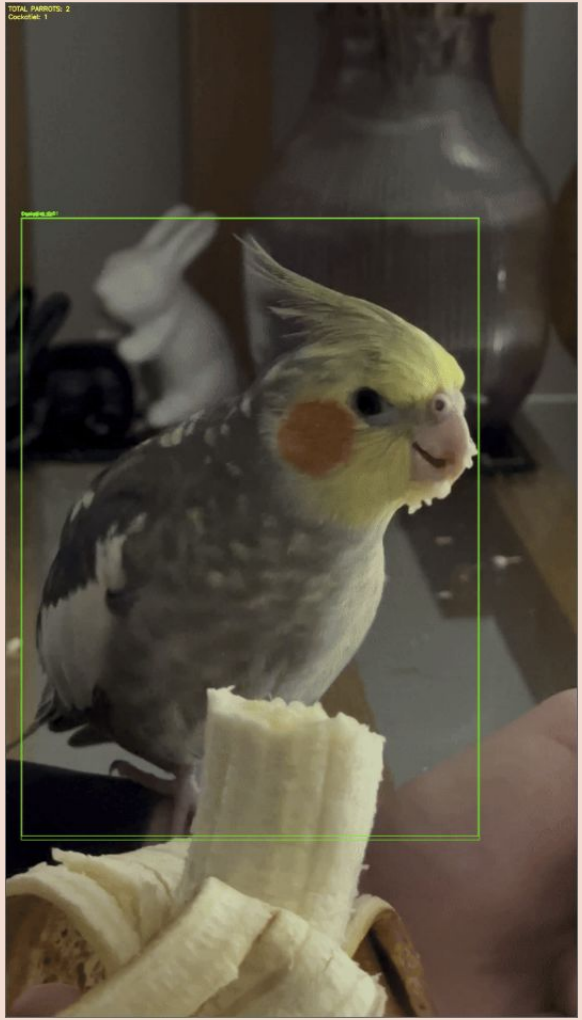
Using the small pretrained detection version (yolo11s.pt) on our dataset

02

Promising results for species detection

- 60 epochs
- mAP50 - 0.91
- mAP50-95 - 0.821


TOTAL PARROTS: 2
Parrot: 0.56
Parrot: 0.96
TikTok
@apolloandfrens



First attempt at pose detection

01

Dual-Model Architecture

- Custom species detection model (previously trained on our dataset based on yolo11s.pt)
- Bird segmentation model from yolo11s-seg.pt → the pixel accurate mask of the parrot (SEG_INTERVAL = 3)

02

Calculated metrics for behavior

- **Displacement**: net distance moved over time (distinguishes "walking" from "fidgeting")
- **Instant speed**: frame-to-frame velocity (detects sudden "flight" bursts)
- **Solidity**: ratio of the bird's area to its convex hull (used to see if a bird is "hunched" while eating)
- **Aspect ratio:** width vs. height (identifies wing-spreading for flight)

Action Classification Rules

Action	Movement Trigger	Overlay Color
Flying	Aspect ratio > 1.4 (wings spread) OR high instant speed	Red
Eating	Low displacement + high solidity (rounded body) + head down	Yellow
Walking	Significant net displacement (> 20 pixels) across the screen	Orange
Sitting	Default state (low movement, standard posture)	Green



Segmentation mask

We tried to detect the pose with a segmentation mask, but the result were underwhelming

Keypoint detection

The next step to be able to find the proper pose was keypoint detection

Keypoints detection

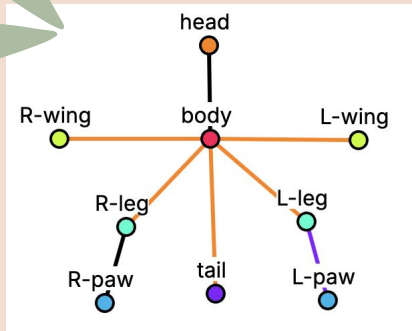
Dataset

No dataset was found for this, so we decided to adapt our dataset for this scope

500 images

Research

Learned about manual labelling, and decided to use Roboflow, adding the keypoints template on the ‘Parrot’ class



Dataset exploration

During the labelling process, we eliminated irrelevant photos and assigned subspecies to all photos

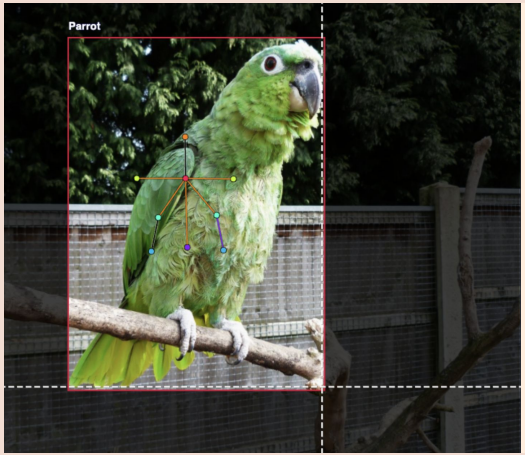
More species

We created more classes (21), since we discovered more species then the initial 18 included

Bird anatomy keypoints

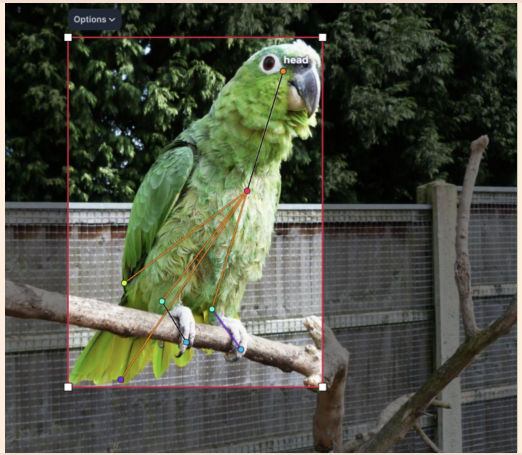


Manual labelling with Roboflow



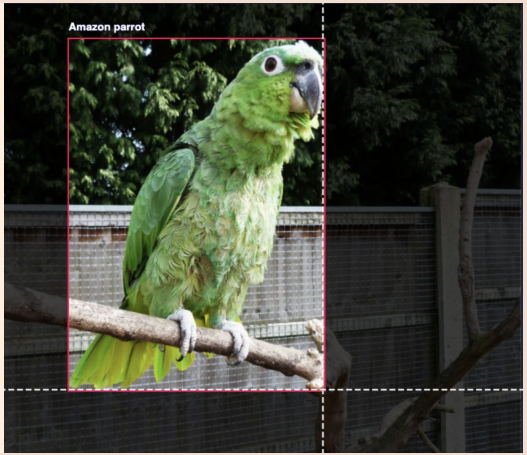
‘Parrot’ class

The image comes with the default skeleton placed in the center of the image



Manual movement

We rearrange the keypoints accordingly, and delete the extra ones



Species assignment

We duplicate the container and assign the species where the image belongs in

Meet the parrot species

African grey parrot



Amazon parrot



Aratinga



Australian king parrot



Budgerigar



Caique



Cockatiel



Cockatoo



Eclectus parrot



Galah



Green cheek conure



Kakariki



Lorikeet



Lovebird



Macaw



Monk parakeet



Pacific parrotlet



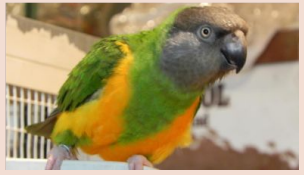
Rosella



Rose-ringed parakeet



Senegal parrot



Model training and results



Model training methodology



We initially began with the goal of creating the most efficient model.

But quickly realised the lack of a big dataset was a problem



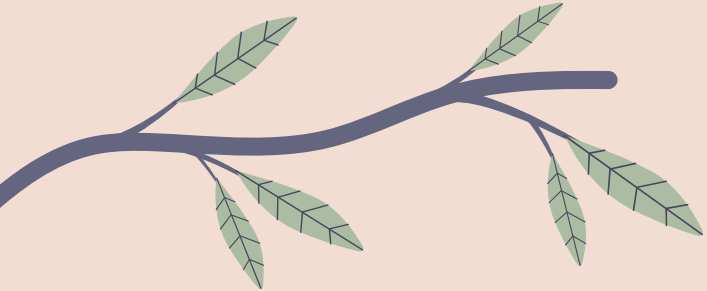


Seeing our dataset evolve (as it was dynamically labeled) made us curious...

How will the model evolve as we feed it more and more data?



Model training methodology



1st

We trained a model with the **default** YOLO 8 params

2nd

Trained a model with custom **online augmentation**

3rd

Trained a model with custom **offline + online augmentation**

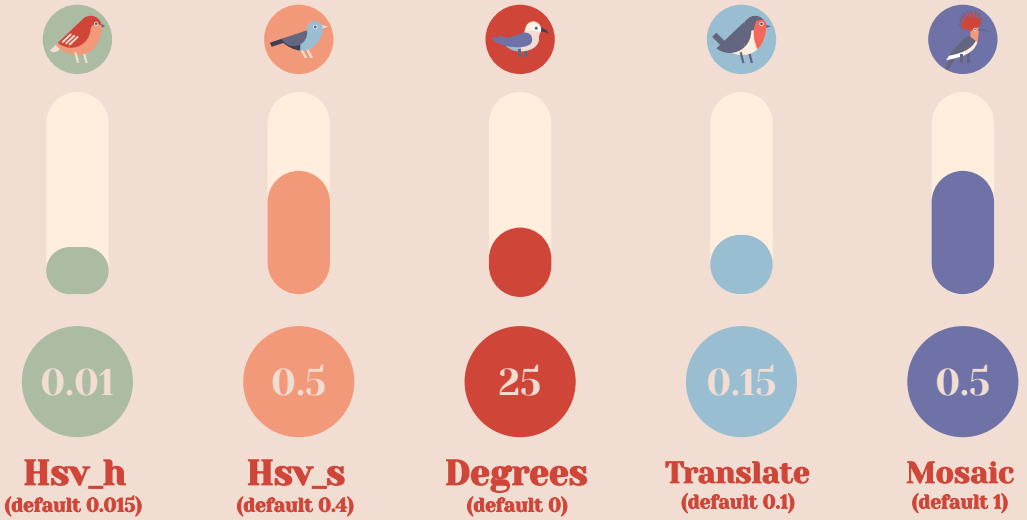
4th (optional)

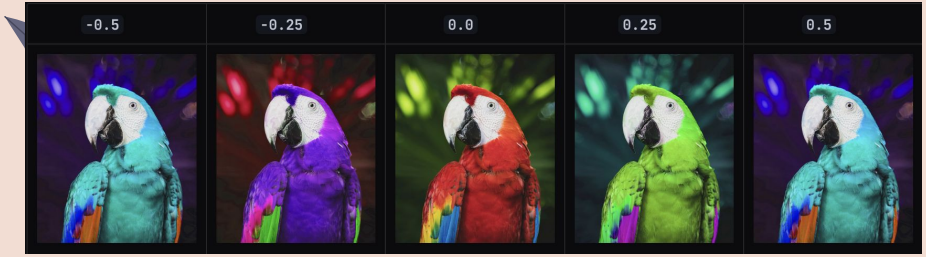
Trained a **YOLO Small** model with **offline + online augs**

Model training methodology
(online augmentation)

As mentioned, we also explored some customization of the augmentations in YOLO.

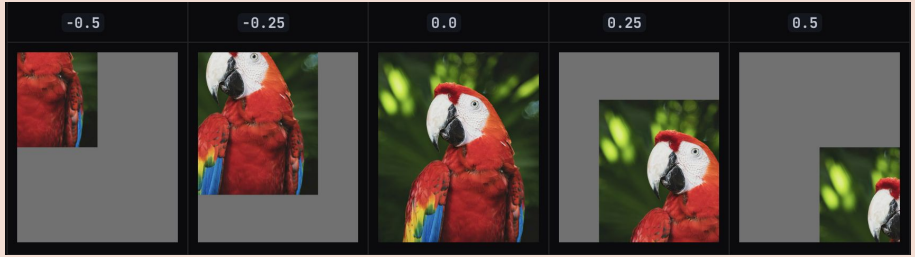
By default, YOLO applies some **online (on-the-fly) augmentations**, but we customized them to fit our parrot detection use case

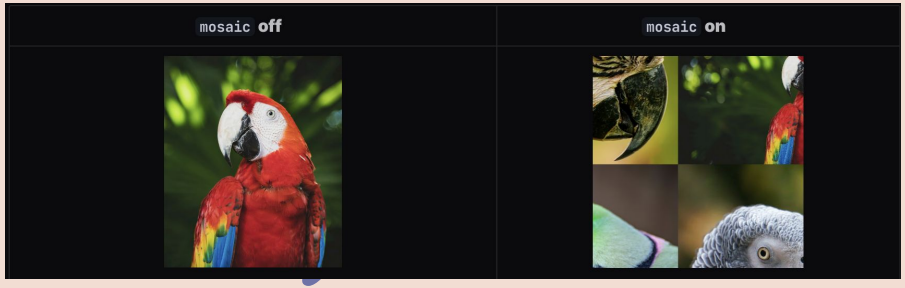



-0.5
-0.25
0.0
0.25
0.5


-1.0
-0.5
0.0
0.5
1.0


-180
-90
0.0
90
180


-0.5
-0.25
0.0
0.25
0.5


mosaic off
mosaic on

Model training methodology

(offline augmentation)

We then moved on to applying offline augmentations, creating new pictures in the dataset with the following augmentations:

Augment	Reason
Blur (0.3)	Simulates motion, focus issues
Color jitter (0.5)	Different lighting, camera settings
Rotation (0.2)	Birds at different angles
Scale (0.5)	Birds at different distances
Noise (0.2)	Low-light conditions, camera quality
Shadow/Fog (0.2)	Outdoor environmental conditions

Model training methodology

(offline augmentation – initial approach)



We initially began by implementing the augmentations by hand, with **mathematical formulas**.

After implementing the blurs (Gaussian, Median and Motion) and comparing them to the ones in the **Albumentations** library, we realized ours is much slower.

Since the code in the Albumentations library is much more optimised (written in **C**, not python), we created a **full pipeline** using it for all the transformations mentioned.



Model training methodology

(offline augmentation)

We also modified some online augmentation setting in YOLO, as we were doing them locally:

Color related

- All were slightly reduced
- Would interfere with the color jitter

Geometric Transforms

- Degrees, translates and scale all slightly lowered
- Shear, perspective and flips up/down disabled





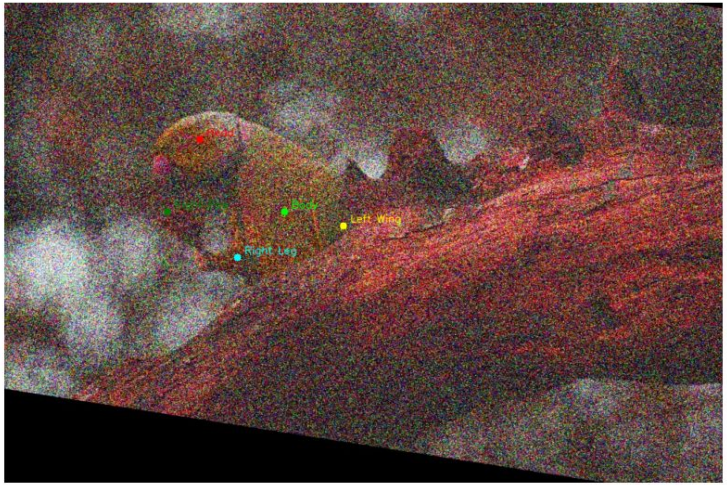
Mixing

- Mosaic reduced and mixup removed
- Goal: we want the keypoints to, so this might interfere

Erasing and copy

- Might interfere with visibility of keypoints and might confuse the model

Keypoints on 0000862a233105cc_jpg.rf.369fa5bd584caf175efef3d8847655fa_aug2.jpg



Keypoints on 0005fd0beaedf55c_jpg.rf.9b707c0d17190cd974bf23aee11146ca_aug3.jpg



Keypoints on 0001b21c9b919d48_jpg.rf.a8cc02489d3fc8fdb2f271147f148a76_aug1.jpg



Model training methodology

Models trained:

	Default	Online only	Offline & Online	Small, Offline & Online
50 images	OK	OK	OK	Too little data!
300 images	OK	OK	OK	OK
500 images	OK	OK	OK	OK

Model training methodology

(extra info)



Dataset Splits

- Were kept as close to 80/10/10 as possible
- Still, test set was unbalanced
- Needed a better way to evaluate models

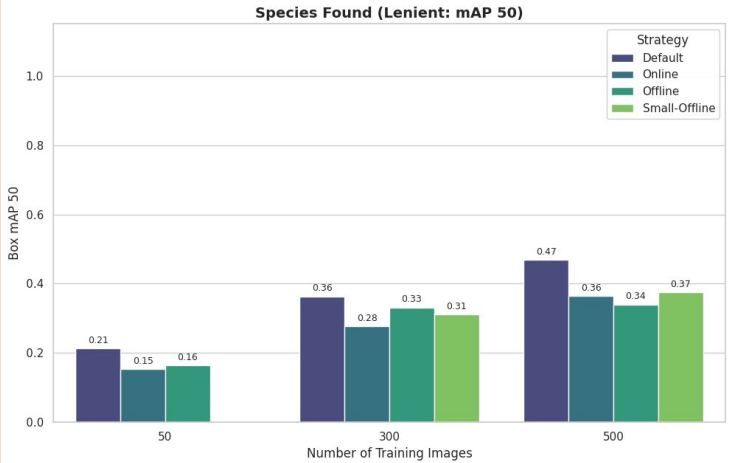


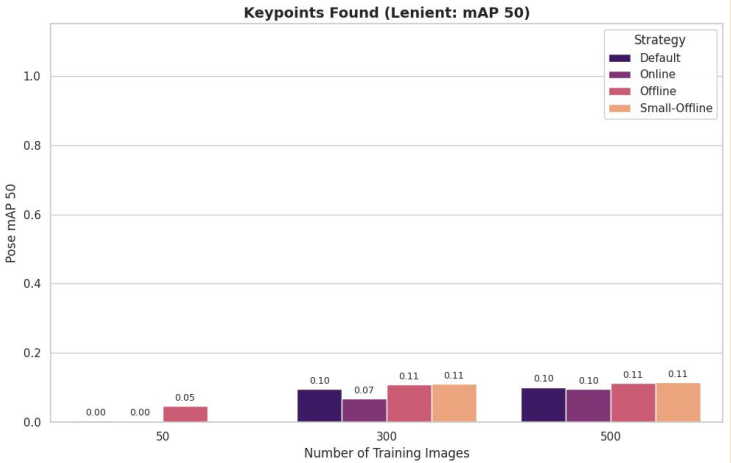
Evaluations

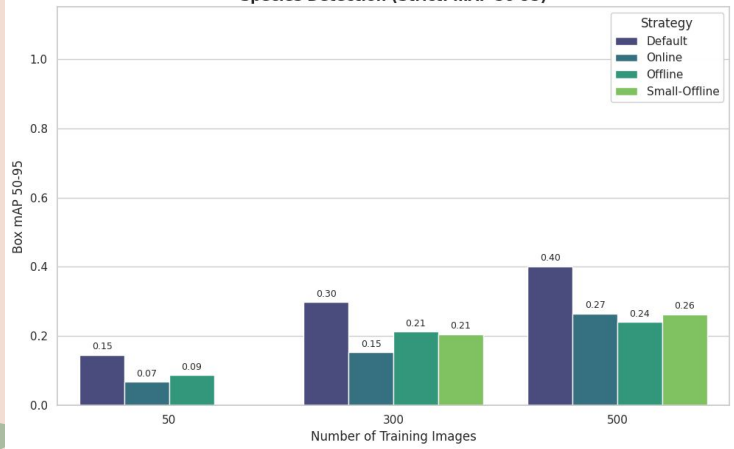
In the end:

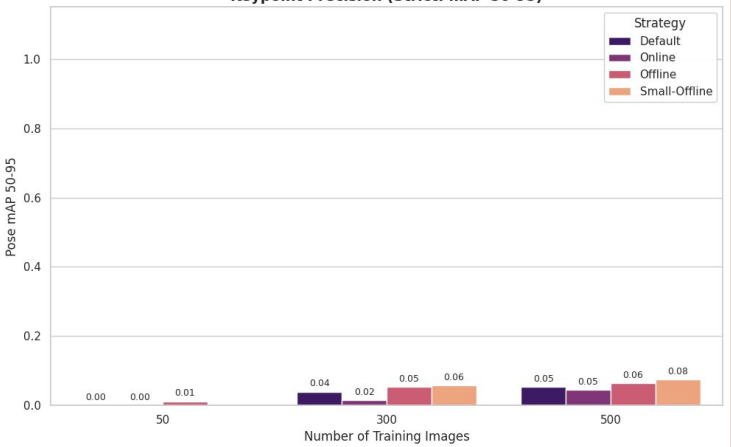
- We took the best epoch of each model
- Used them to predict 50 never-seen labeled images and measured performance

YOLOv8 Parrot Detection: Performance Dashboard (Including Small Models)

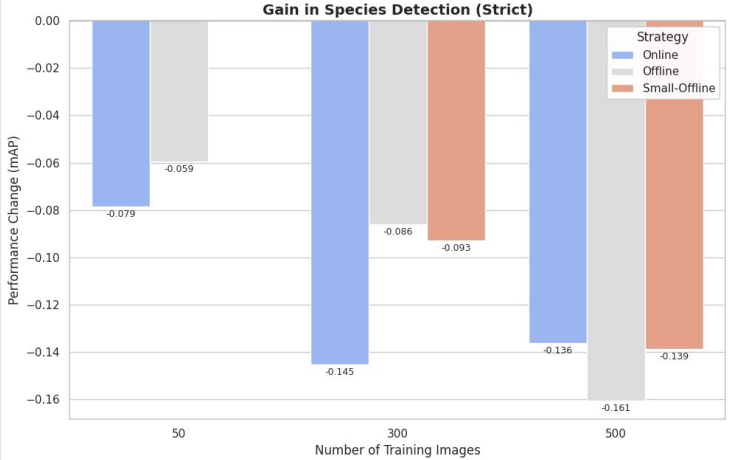


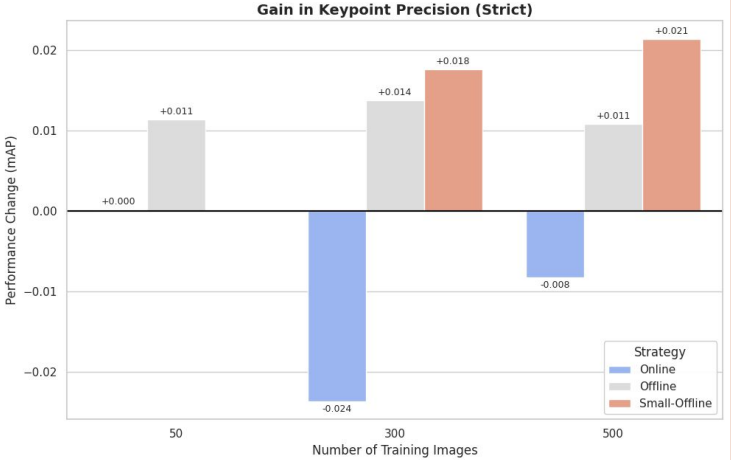


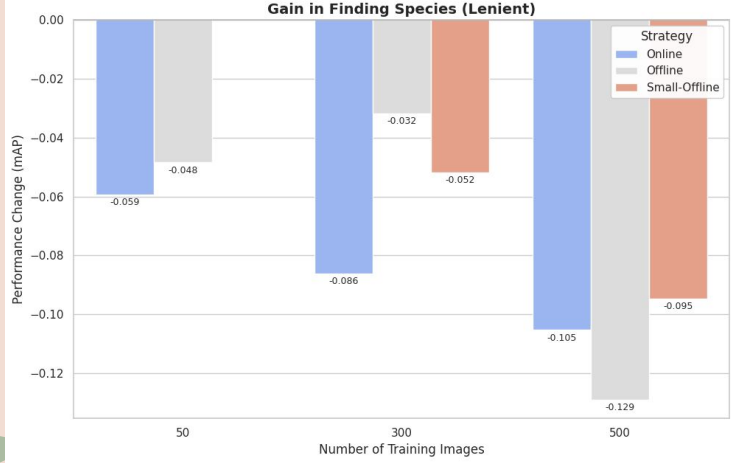


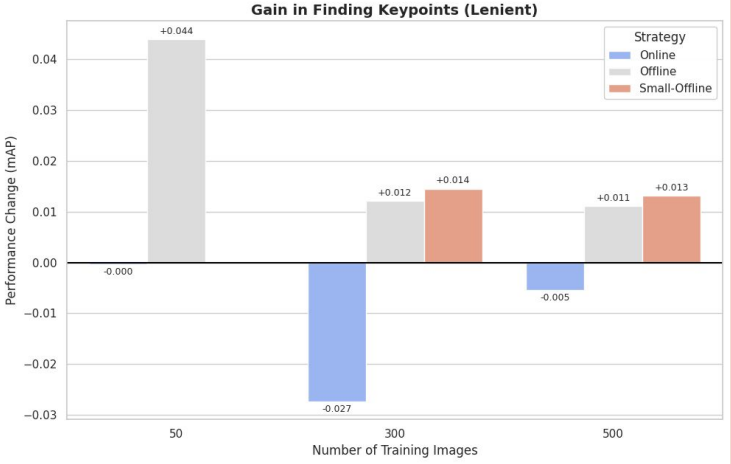


Impact vs. Nano-Default Baseline



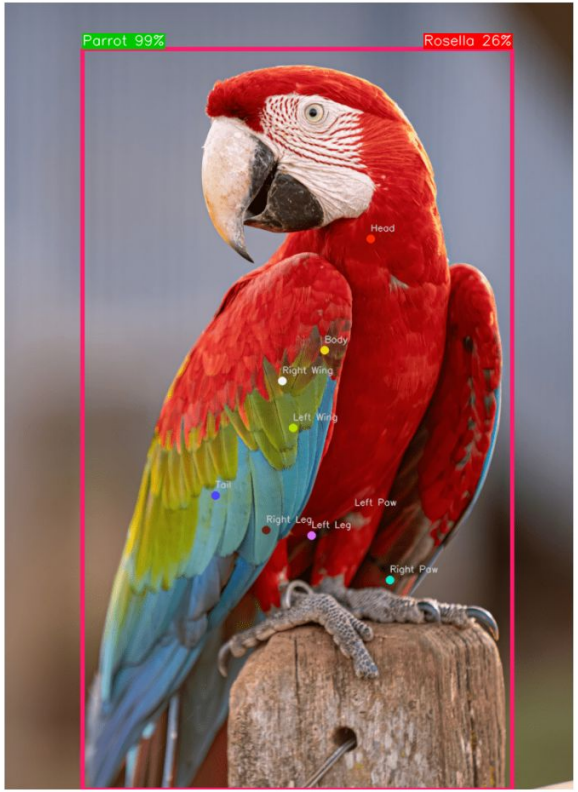


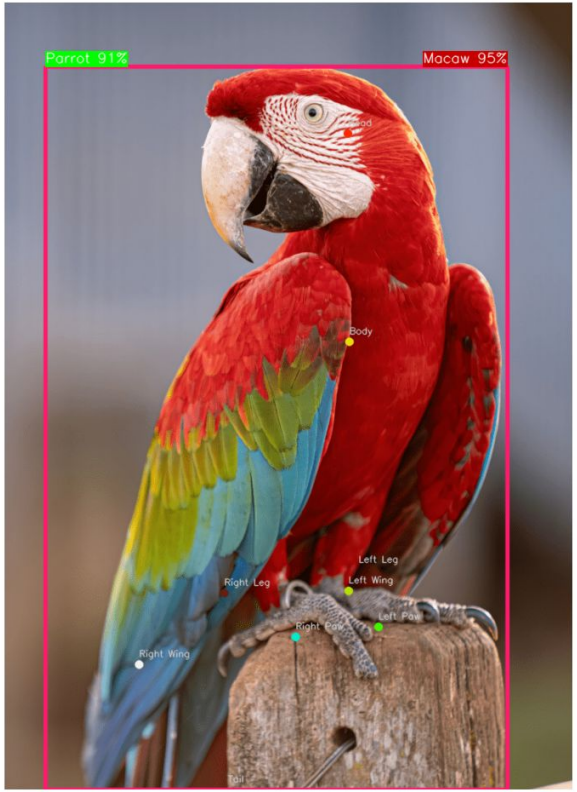




Model training results

(simple image: 50 default VS 500 default)

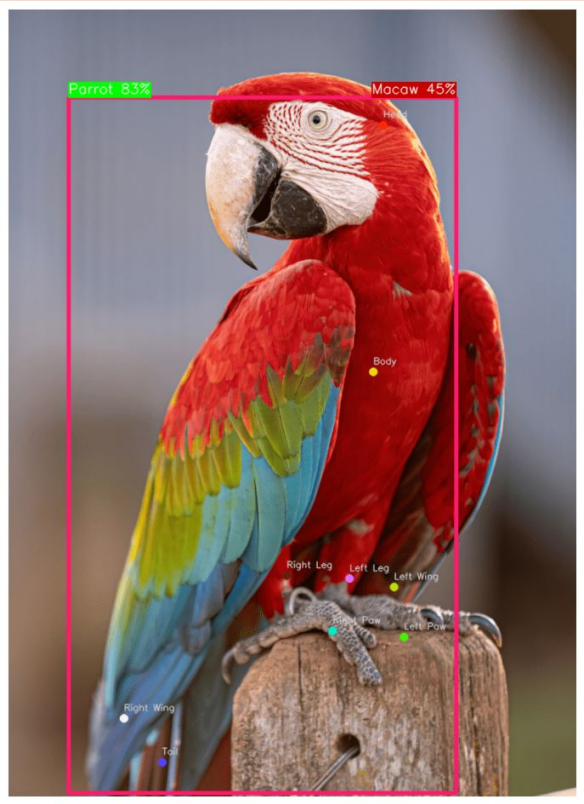




Model training results

(simple image: 500 default VS 500 augmented small)

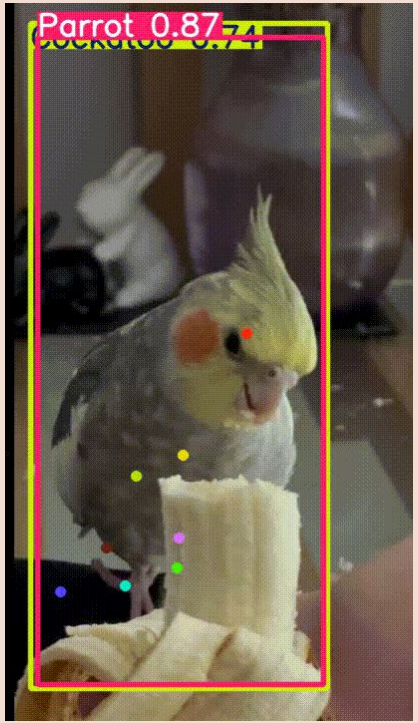




Model training results

(Winnie Video: 50 default VS 500 augmented small)





DeepLabCut Experimentation

We have also experimented with DeepLabCut: a markerless pose estimation framework



Extracted

Used DLC's Automatic Frame Extraction (k-means clustering) to select distinct frames for labeling.



Annotated

Manually labeled keypoints (Head, Wings, Body, Feet) on the extracted frames.



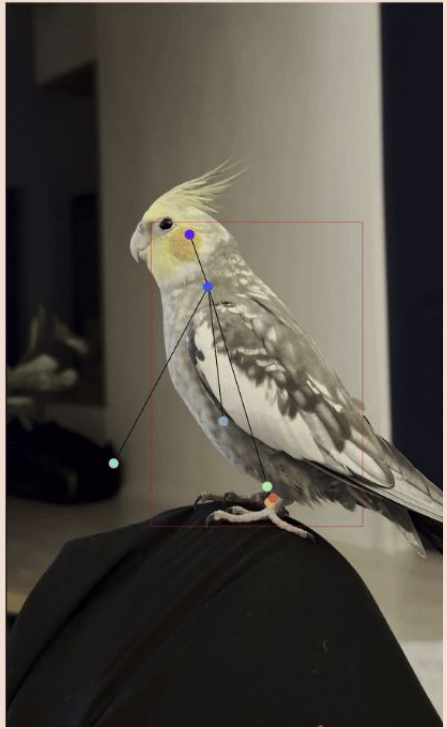
Trained

Trained a few models (from ImageNet and Fine Tuned a SuperAnimal Bird model)

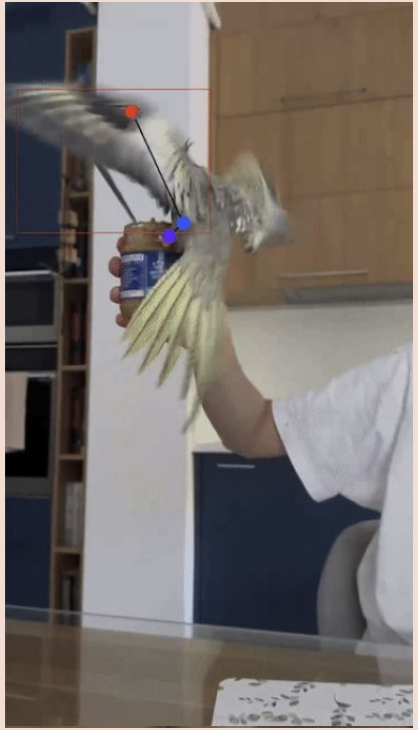


Analyzed

Got some nice results, but there was a lot to experiment with

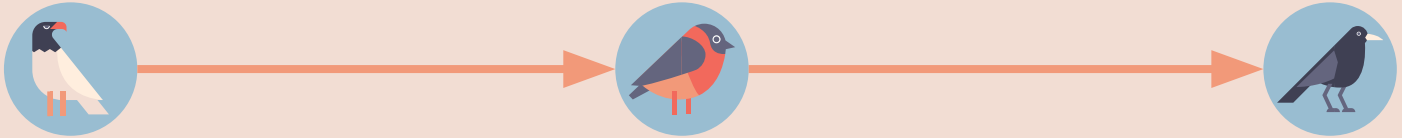


Simple video results were good



Complex ones'... not so much

Future work



YOLO 11

Make the experiments with a newer Ultralytics model, compare results

Posture detection

Based on the best performing keypoint detection model, we'll try to determine what a parrot is doing in a video (e.g. flying, preening, eating)

More labelling

Only managed to label 500 out of the 2500 images in the dataset
more data=better results

References

Additional experiments

Using ‘DeepLabCut’



Main inspiration

‘Posture recognition for birds based on YOLOv8 keypoints regression’

Ultralytics YOLO


Thank you!